N=96: the transition is measurable — and the seed beats depth

6 cells, 67 GPU-h · jobs 50860571 + 50903022 · commit d016168 · rho_50 = hint density at solve rate 0.5, LOWER IS BETTER

H1 PASSES — the measurement works, which it did not at N=48.

All six cells cross solve_rate 0.5 strictly inside the rho grid, so rho_50 is an interpolation and not a censored bound. At N=48 every cell solved the whole grid and rho_50 could only be quoted as '< floor'. dd_mean_T runs 1.2 to 21.3 rounds down to a floor of 0.325: fast phase to deep slow phase.

H4 FAILS at 2.8x its bar — and the pre-registration says that voids H2 and H3.

Two pairs differ ONLY in --seed. Their rho_50 spreads (8: 0.138, 12: 0.115) both blow past the 0.05 bar, with paired per-rho solve differences peaking at 0.490. The sbatch header committed the reading in advance: 'if it is larger than 0.05 here, this whole grid is one draw and only the ordering survives.' H2's L=8->12 gap is +0.012, H3's margin 0.069 — neither is a measurement.

WHY — rho_50 is essentially a function of how far the run converged.

val_nll rank-orders rho_50 across all six cells at Spearman +0.943; 0.086 of val_nll spans 0.266 of rho_50; all six are still descending when training stops. The seed moves val_nll by up to 0.065, the whole L=8->16 depth range by 0.018. Depth, data and seed are three ways of moving one number, and the seed moves it ~4x further.

Best rho_50 improved 0.691 -> 0.564 on the same eval. It is a seed draw, not a method.

The one-knob decomposition puts 90% of that on the seed, 10% on depth. The seed cuts both ways and the downside is larger: 0->1 bought -0.115 at L=12 and cost +0.138 at L=8, making t5 L=8 240k s1 the grid's worst at 0.829. 50933500 (cosine, one knob, two seeds) is running to test whether the spread can be reduced at all.

Every number read at build time from the committed reader scripts/n96_fallscan_analysis.py, the same one the results notebook imports. Eval: 14-point rho grid 0.95-0.32, 400 fresh instances per bin, identical instances across all six cells, so every cell-to-cell difference is paired.

What the model actually sees

Every token is one categorical id into a 106-symbol vocabulary (N=96, q=4, degree=5). No text, no digits-as-characters.

VOCABULARY

0-5	`BEGIN_HINTS BEGIN_GRAPH BEGIN_REASONING`
	`BEGIN_ROUND SOLVED UNRESOLVED`
	markers (UNRESOLVED = DD got stuck; unused here)
6-9	`COLOR_0 COLOR_1 COLOR_2 COLOR_3`
	the q colours (token name COLOR_n, prose 'colour')
10-105	`N0 N1 ... N95`
	one id per node, no structure in the id

GRAMMAR

`BEGIN_HINTS`	`(NODE COLOR)*`
`BEGIN_GRAPH`	`(NODE (NODE){degree})*`
`BEGIN_REASONING`	`(BEGIN_ROUND (NODE COLOR)+)*`
`SOLVED`	

*regex: * = zero or more, + = one or more, {degree} = exactly 5*

The graph section is a neighbour table, not an image: node, then its 5 neighbours.

THE ORDER WITHIN A ROW IS THE LEAK.

THIS INSTANCE, 790 TOKENS

HINTS	`BEGIN_HINTS N1 COLOR_0 N3 COLOR_1 N5 COLOR_0 N10 COLOR_2 N14 COLOR_0 N18 COLOR_0 ... (71 tokens)`
GRAPH	`BEGIN_GRAPH N0 N34 N73 N78 N19 N80 N1 N37 N56 N31 N36 N61 ... (577 tokens)`
REASONING	`BEGIN_REASONING BEGIN_ROUND N6 COLOR_3 N7 COLOR_1 N9 COLOR_1 N19 COLOR_1 N22 COLOR_1 N28 ... (142 tokens)`

TRAINING LOSS is next-token cross-entropy on EVERY token, the problem statement included -- there is no section mask.
AT EVAL the model is PROMPTED with everything through BEGIN_REASONING and generates the rest: 18 rounds, 61 deductions,
122 node+colour tokens, plus the BEGIN_ROUND markers and the SOLVED/UNRESOLVED terminator, which are trained targets like any other token.

DD is single-candidate elimination -- sudoku's 'naked single' rule transplanted to graph colouring, which is where the code's sudoku names come from; the task itself is the graph, not a 9x9 grid. Instance shown: seed 50000000016, FALL-colouring ensemble (every node sees all q-1 other colours, so no node is permanently undeducible).

One real reasoning trace

35 hints of 96 nodes, then 18 rounds of simultaneous deduction. Hints: N1=COLOR_0 N3=COLOR_1 N5=COLOR_0 N10=COLOR_2 N14=COLOR_0 N18=COLOR_0 ...

THE TRACE, AS EMITTED

```
BEGIN_ROUND        N6 COLOR_3 N7 COLOR_1 N9 COLOR_1 N19 COLOR_1 ...
  13 deduced
BEGIN_ROUND        N0 COLOR_0 N16 COLOR_0 N42 COLOR_3 N44 COLOR_1 ...
  6 deduced
BEGIN_ROUND        N4 COLOR_1 N11 COLOR_3 N55 COLOR_2 N69 COLOR_3 ...
  5 deduced
BEGIN_ROUND        N2 COLOR_2 N15 COLOR_0 N58 COLOR_0
  3 deduced
... 14 further rounds

SOLVED
```

WHY ONE DEDUCTION FIRES

node N6, neighbours N76 N12 N80 N89 N87

their colours known before this round:

```
[0, 1, 2]  (3 = q-1 of the 4)

(3 known neighbours, 3 DISTINCT colours)

still unknown: N76 N12
```

only colour left: 3

```
so the trace emits  N6 COLOR_3
```

The model is never shown this reasoning -- only the node/colour pair.

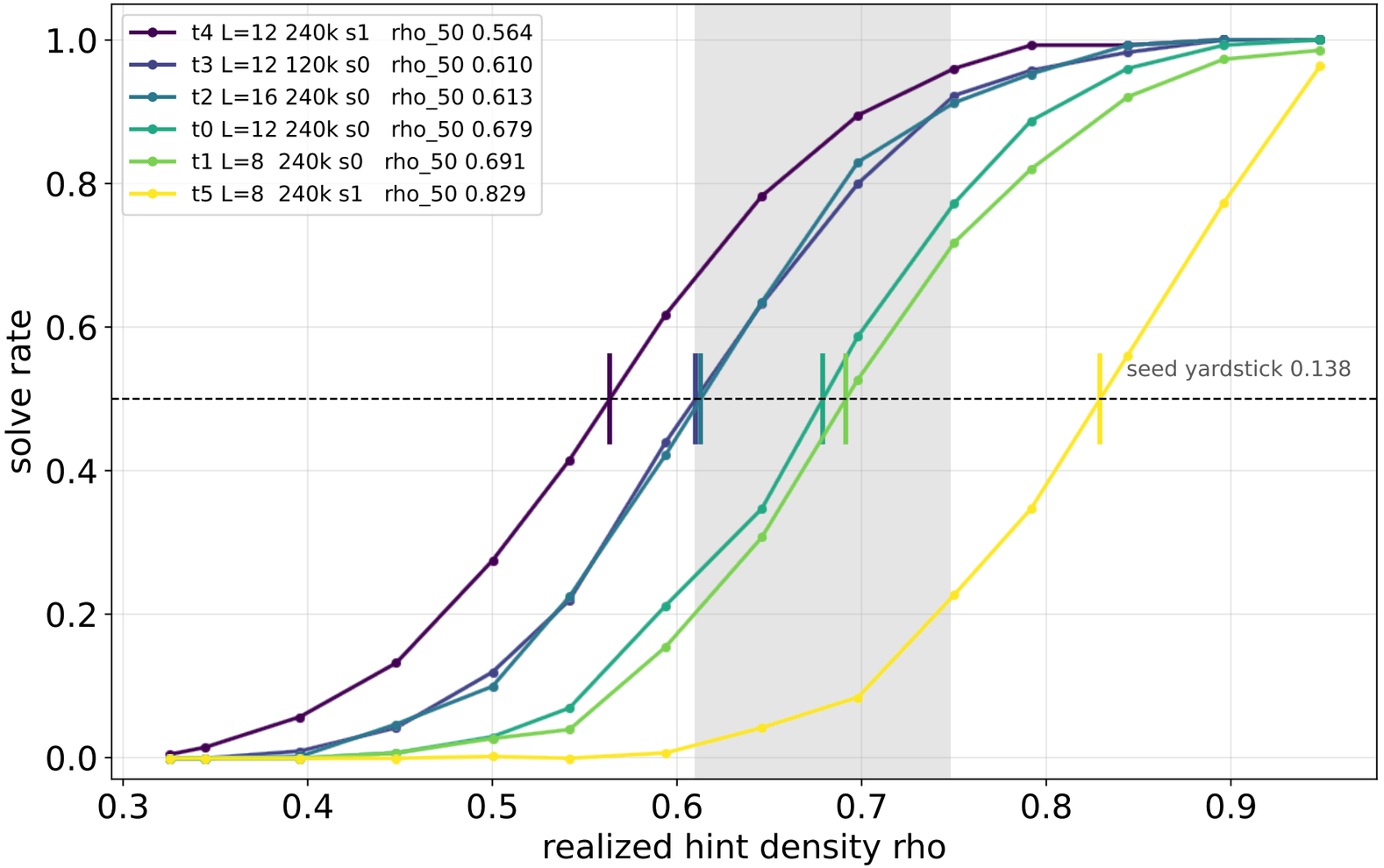
A round deduces every node whose neighbours already cover q-1 colours, all at once. Reasoning depth T is the number of rounds: a deduction in round 3 depends on ones made in rounds 1 and 2, which is the serial computation that network depth has to supply.

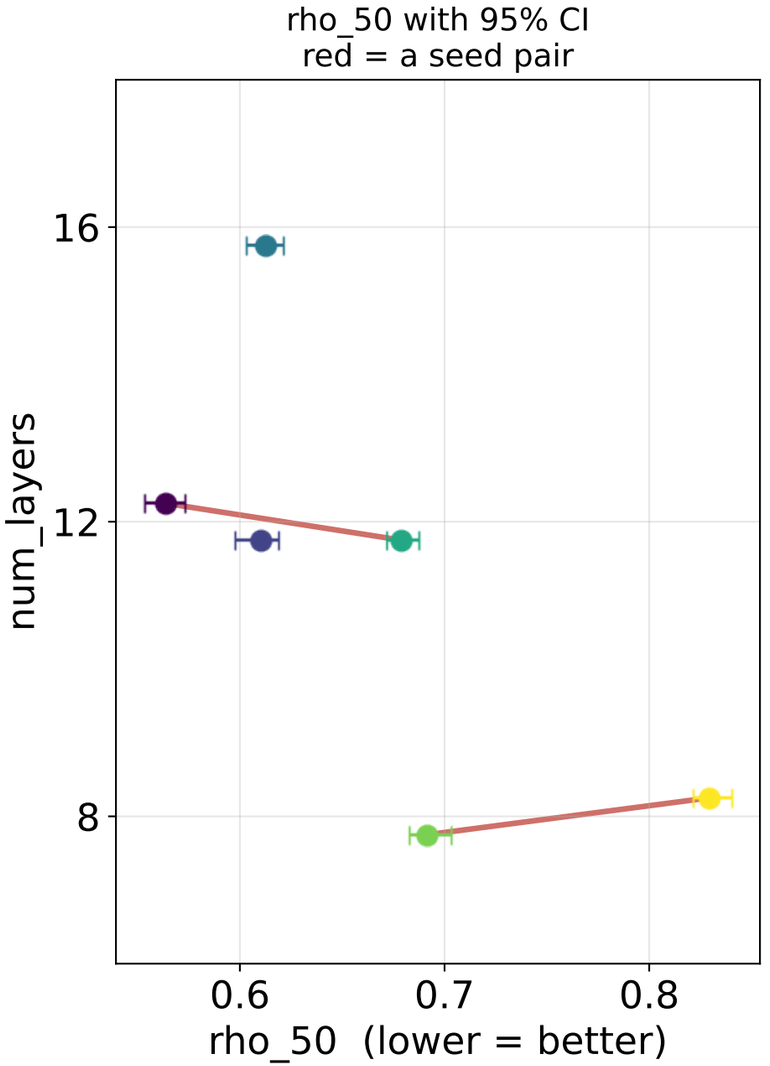
N=96, degree 5, 4 colours, --shuffle-adj --fall: the task all six cells in this deck were trained on. rho=0.33 is the deep end of the eval grid, below where any cell still crosses 50% success.

T=18 rounds, the 90th percentile of 40 draws from this config (range 1-23) -- one instance, shown to make the task legible; no result in this deck is computed from it. The worked deduction on the right uses the state BEFORE this round, which is what DD itself saw; it is recomputed from the graph, not read out of the model.

Success rate across the DD transition, all six cells

400 paired instances per point. Ticks mark rho_50; the grey band is what ONE SEED is worth.



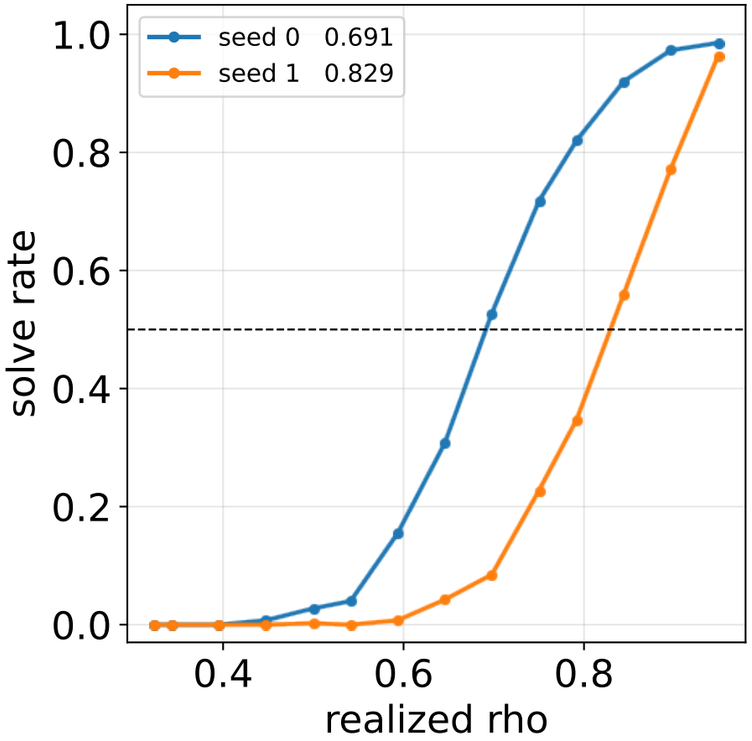


CIs are binomial resampling of each bin (n=400) pushed back through the same interpolation, so they are SAMPLING error only — about +/-0.01, roughly a tenth of the seed term. Quoting them as the uncertainty on an architecture would understate it by an order of magnitude; that is the whole lesson of this deck.

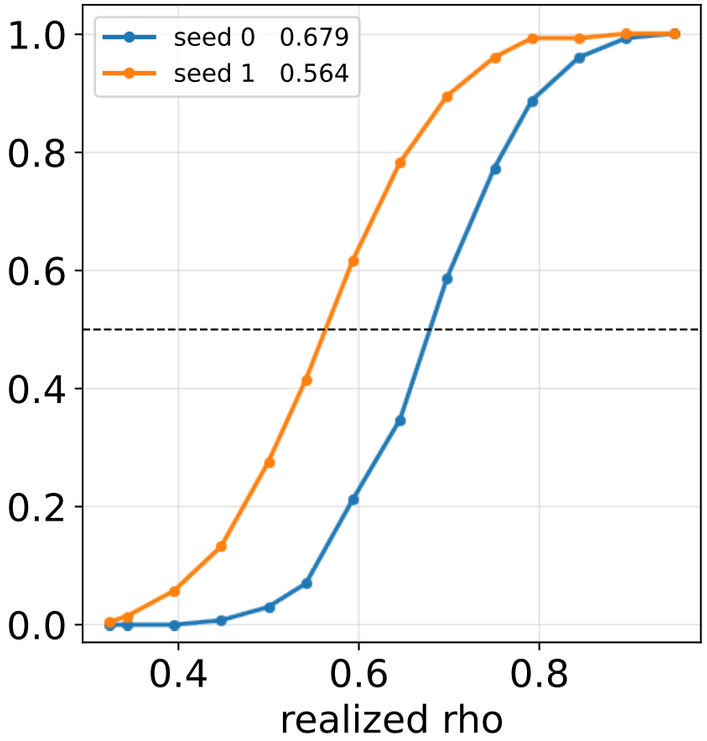
The control that broke the grid: same config, different seed

Each pair is byte-identical but for --seed. Pre-registered bar: spread <= 0.05.

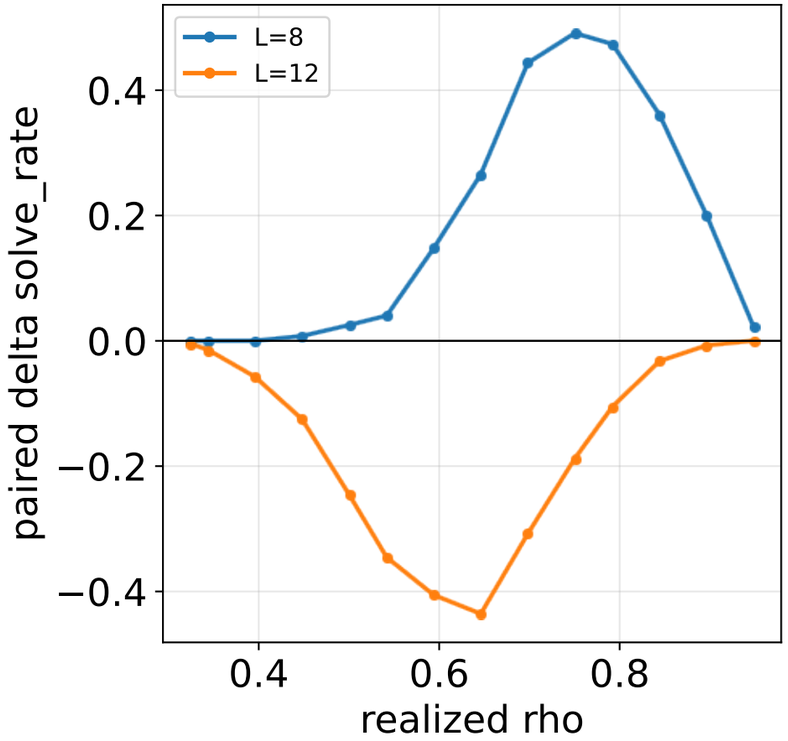
L=8, 240k
spread 0.138 (FAIL)



L=12, 240k
spread 0.115 (FAIL)



seed 0 - seed 1, same instances



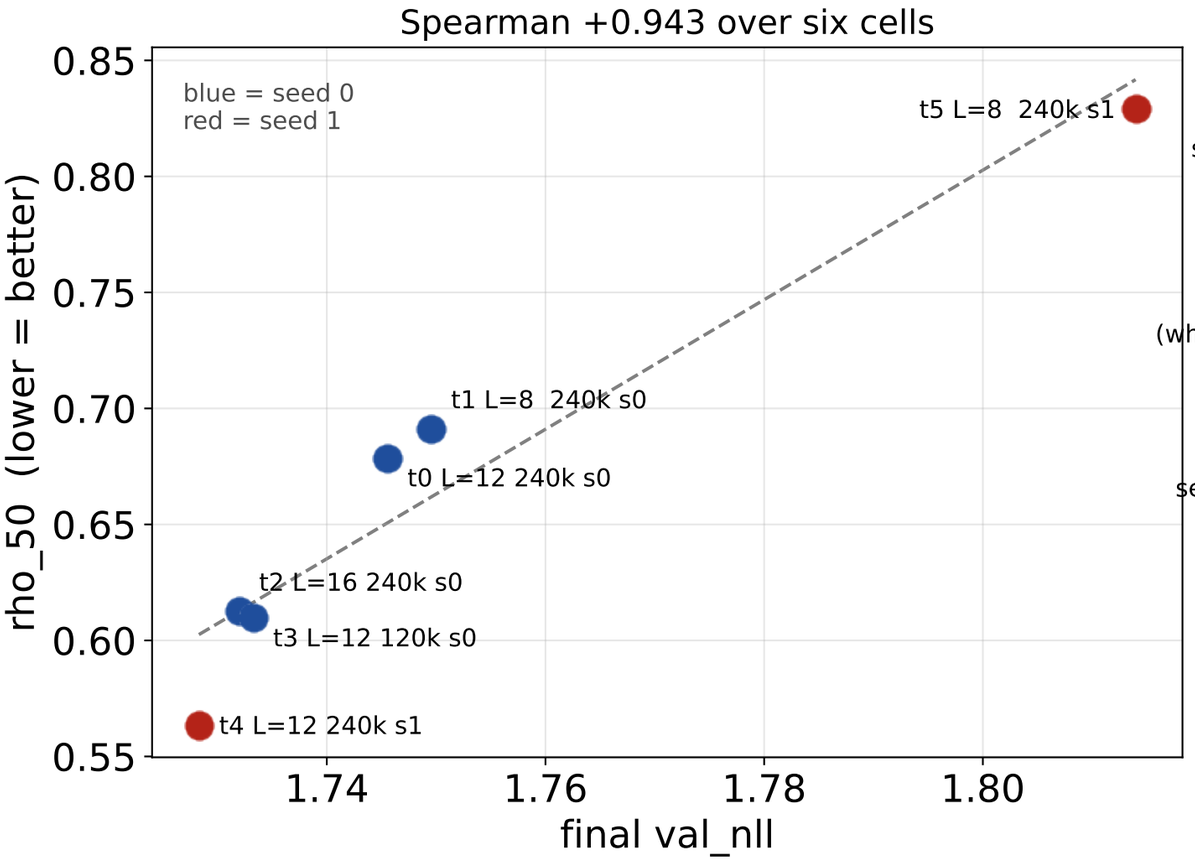
Yardstick in force: 0.138 — the larger of the two spreads: a gap is only as sharp as its noisier rung.

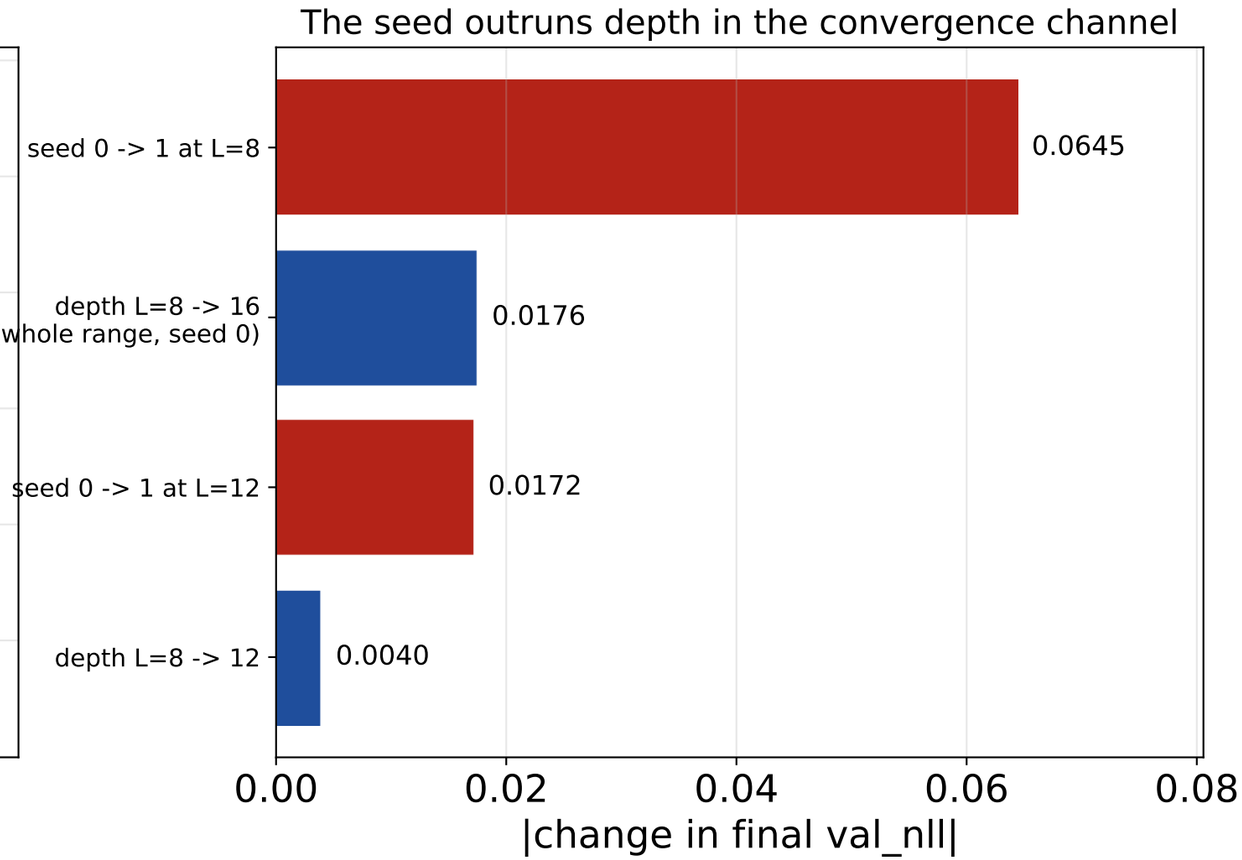
The L=8 replicate (50903022) was submitted precisely in case the shallow end was quieter and could rescue H2.
It is the noisier end, and it set the yardstick — reproducing finding 41, which measured the big seed term at the shallow end at N=48.

These are paired curves: both seeds of a pair see the same 400 instances at each rho, so the difference between them is the model, not the sample. The peak paired difference is 0.490 of solve rate at L=8 — half the dynamic range of the metric.

What is actually varying: how far each run converged

Finding 4 put val_nll vs solve rate at r=+0.33 ACROSS models. Within one N, one ensemble and one protocol it is almost deterministic.





Consequence 1: inside a fixed (N, ensemble, protocol), val_nll is a near-free proxy for rho_50 — the 14-point x 400-instance eval costs hours per cell, and the ranking is already sitting in history.json.

Consequence 2: a lever that lowers val_nll and tightens its spread should move rho_50 with it. That is the premise of 50933500, and the reason the planned depth scan is blocked behind it.

All six cells still have val_nll falling at the last epoch (last-5 drop negative everywhere), so none is capacity-bound — they are all stopped mid-descent, which is exactly the regime in which where the iterate happens to sit is a lottery. The negative train-minus-val gap is finding 16's averaging artifact, not underfitting.

Before and after on the same eval — and why the next run is not a depth scan

No earlier N=96 curve exists — this eval starts with 50860571. 'Before' is the only cell that had landed when asked.

BEFORE t1 L=8 240k s0 rho_50 0.691 -> AFTER t4 L=12 240k s1 rho_50 0.564 (-0.128, CIs disjoint)

The after-cell differs from the before-cell in TWO knobs, depth 8->12 and seed 0->1. The one-knob depth control lands within 0.012 of the before-cell, so 90% of the move is the seed. Had the other L=8 seed landed first this morning, the same procedure would have reported 0.829 as the best. No method improvement is claimed.

Averaging the seed away is not affordable: ~94 seeds per rung, ~2,068 GPU-h.

At sigma_hat 0.122 per cell, resolving H2's own 0.05 bar at 95%/80% needs ~94 seeds at each of two rungs, ~2,068 GPU-h against the ~67 this whole grid cost. The completed grid also closes the escape route: the spread is not a quirk of L=12, it is LARGER at L=8. The seed term must be reduced, not sampled.

RUNNING NOW — 50933500, one knob, two seeds, pre-registered.

--lr-schedule cosine at L=12 / 240k / 50 epochs, seeds 0 and 1, otherwise identical to the pair above. Every cell here stops mid-descent at constant lr, so the end-point is a lottery; decay should lower val_nll and tighten its spread. P1, the test: does the spread fall below 0.115? P3, beating 0.564, is NOT the test — 0.564 is itself a lucky draw. Falsifier: spread >= 0.115 AND no val_nll gain means the seed term is intrinsic at N=96, and that is the finding, not a third schedule.

What H2 becomes if the spread does come down.

The depth ordering is right — 0.691 -> 0.679 -> 0.613 as L goes 8 -> 12 -> 16 — and the 12->16 step (+0.066) is the largest architectural effect here, about half the yardstick. So the cheapest real test of depth is L=12 vs L=16 UNDER the schedule, not a rerun of the 8->12 gap that failed. That is the step the standing depth question needs, and it stays blocked until the spread is known.

Deck and results notebook both read scripts/n96_fallscan_analysis.py at build time. Notebook: notebooks/sudoku_n96_fallscan_2026-11-03_results.ipynb. Curves and checkpoints: $SCRATCH/spin_gnn/sudoku_n96_fall_scan_2026-11-02/.